Structured export

LW Registry Project Visual Brief

A cleaned, visual PDF version of the project review conversation. This report explains how the platform is structured, how the system connects, how each role moves through it, and how the modules link together.

What this PDF contains

- System architecture and stack
- Platform connection map
- Public, SBO manager, and admin flow maps
- Module landscape and dependencies
- Workflow and event connections

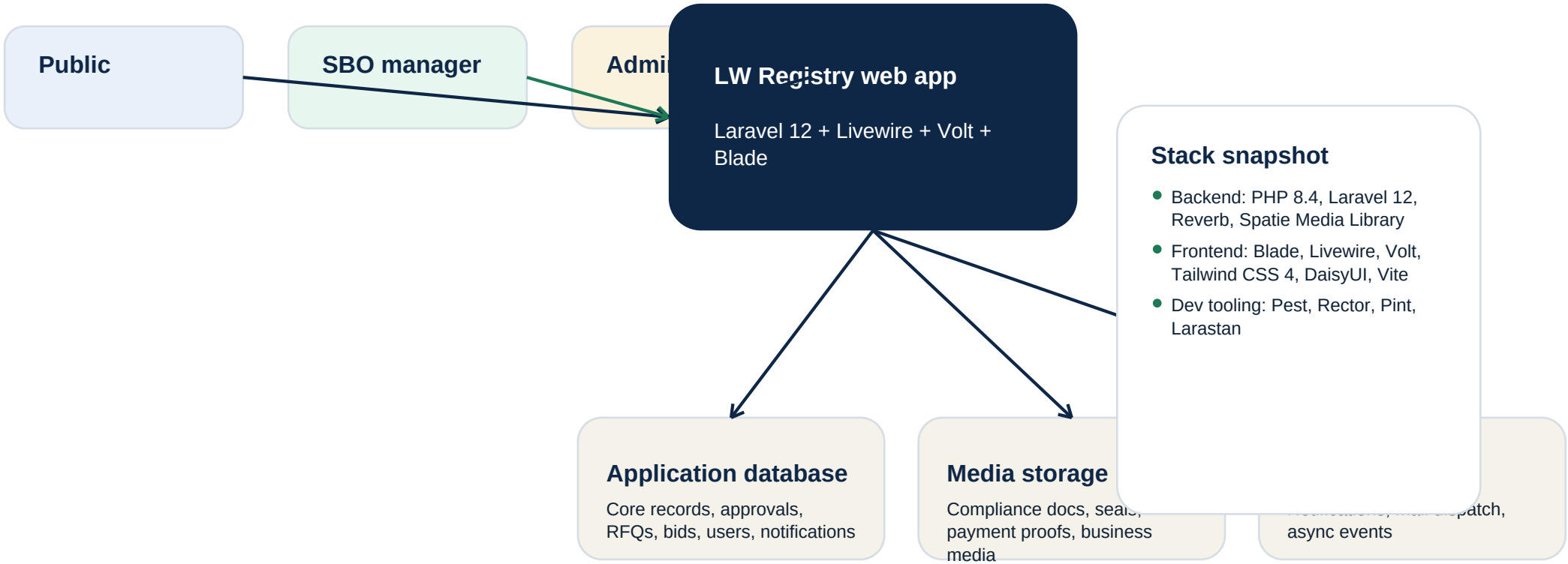
Executive summary

- The platform is already a substantial Laravel 12 web system, not a prototype. It handles public discovery, SBO onboarding, payments, procurement, support, and admin operations.
- Its architecture is centered on one application, one database, one media store, and queued mail/notification flows.
- Each role enters through a different route, but the workflows converge on the same approval, payment, document, and verification core.
- The module structure is capability-based, so the clearest way to understand the platform is to read it as connected journeys rather than isolated screens.

Sources: PRODUCT.md, docs/platform-architecture.md, composer.json, package.json

System at a glance

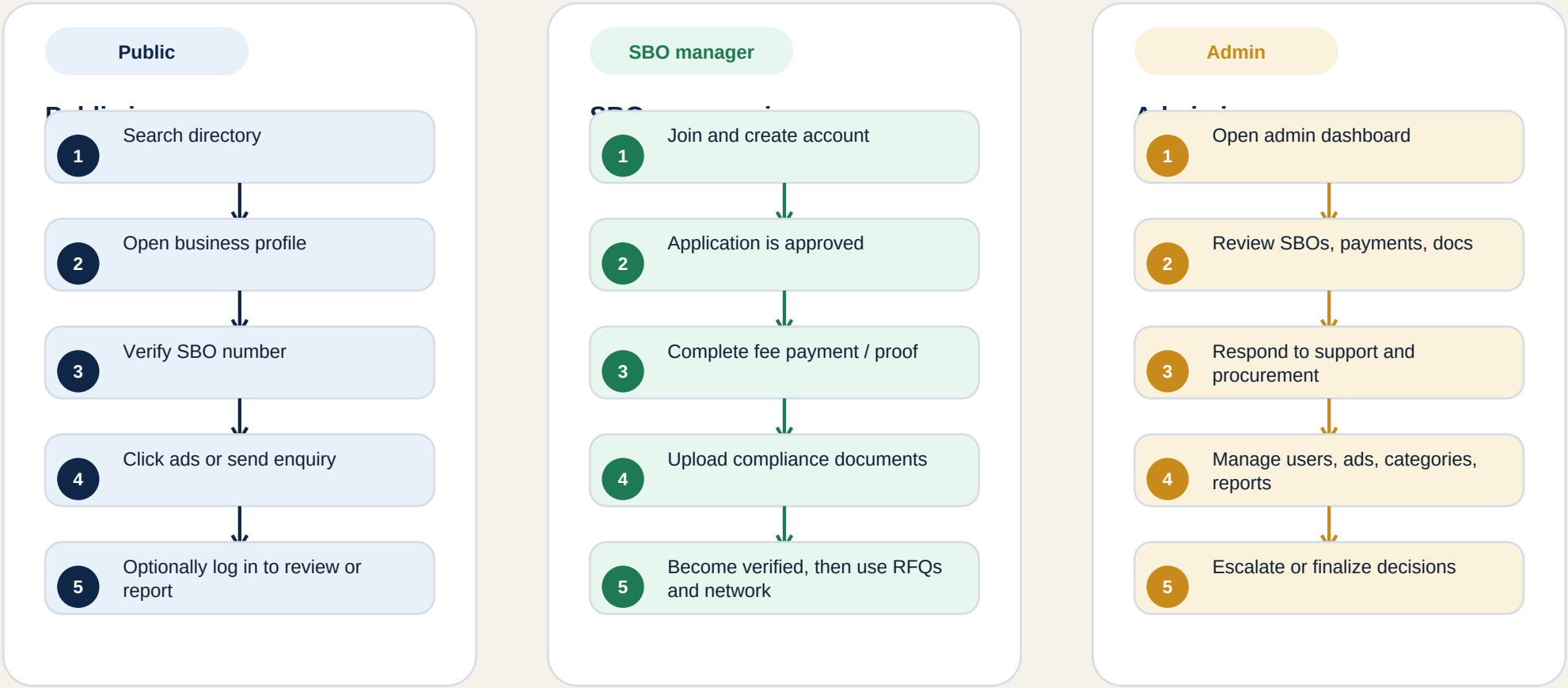
The project works as one connected web platform: roles come in through the app, the app writes to the database and files, and queued notifications push outward.



Sources: docs/platform-architecture.md, composer.json, package.json, routes/web.php

Role-by-role flow map

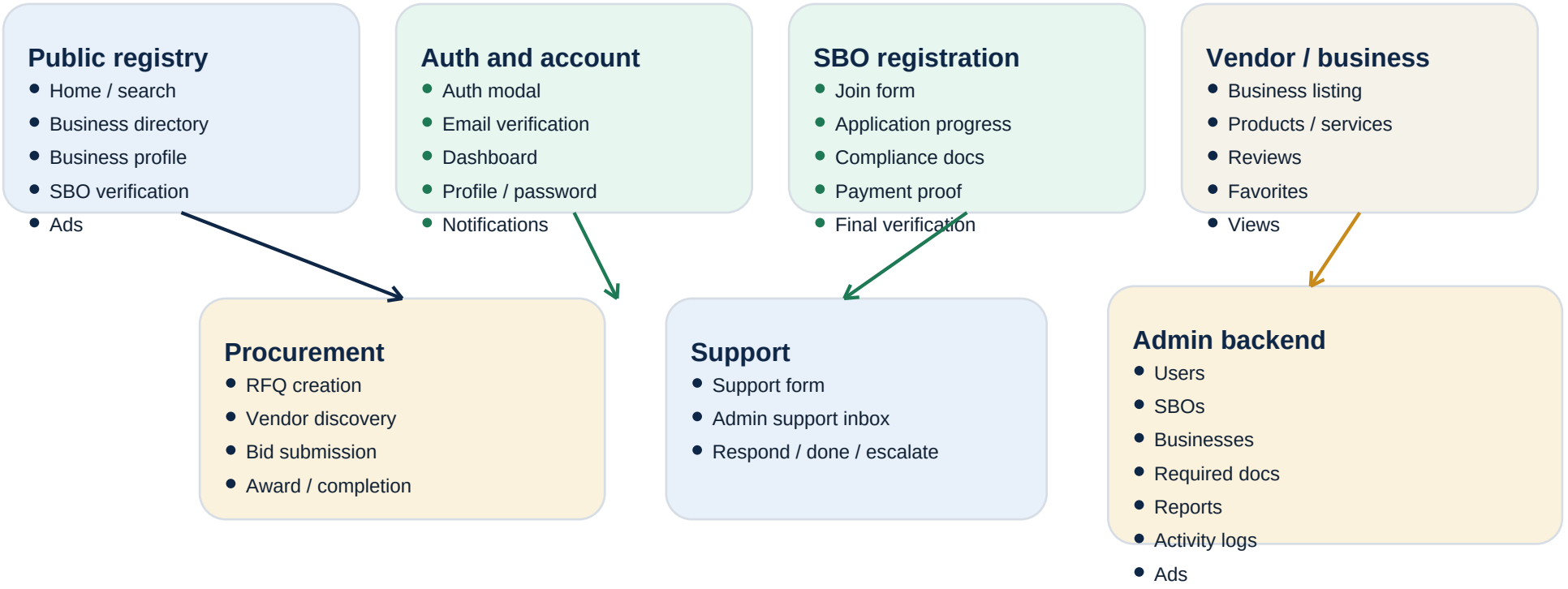
The same platform serves three very different journeys. The main difference is the entry point and the workflow rules applied after login.



Sources: routes/web.php, SboApplicationProgress.php, SBO dashboard/application views

Module landscape

The codebase is organized around business capabilities, not isolated microservices. Most future API work can follow these same module boundaries.



What this means: the platform already has clear functional seams, those seams can map to API resources later, and most coupling today is workflow- and state-driven rather than service-driven.

Sources: docs/platform-architecture.md, routes/web.php

How the platform works end-to-end

The live code path is: register, approval, payment, document compliance, verification, then wider platform access.

1. **Join**
   User creates account and submits SBO registration
2. **Approval**
   Admins review the application and approve it into the paid workflow
3. **Payment**
   Fee is confirmed through Espees or manual proof review
4. **Documents**
   After payment confirmation, the owner uploads required compliance files
5. **Verification**
   If all conditions are met, the SBO becomes verified
6. **Operate**
   Verified SBO can post RFQs, join the network, and appear as trusted

After verification

- Public users can trust the listing and verify the MOC number.
- The SBO manager can access procurement and network tools.
- Admins continue governance through support, reporting, and approvals.

Why this matters technically

- Verification state drives access control.
- Payment confirmation is the gate that unlocks the document stage in the live UI.
- Documents, payment, and admin review are not separate features; they are one workflow.
- Any API or rebuild path must preserve that same state machine.

Sources: SboApplicationProgress.php, sbo-fees.blade.php, SboRegistrationReceivedNotification.php, SboUploadDocumentsNotification.php, routes/web.php

How the platform stays connected

Routes, shared workflow state, and notifications are the main glue between modules. That is what makes the system behave like one platform instead of separate pages.

Entry points

Entry points

- Public routes: /, /businesses, /business/{slug}, /verify/sbo/{number}
- User routes: /dashboard, /profile, /notifications, /support
- SBO routes: /sbo/dashboard, /sbo/applications, /my-rfqs, /sbo/network
- Admin routes: /admin and capability-based admin pages

Shared workflow state

Shared workflow state

- Email verification gates the post-login router
- Role checks redirect users into the right workspace
- SBO fee/payment state controls progression
- Document compliance and verification state unlock later modules

Event and notification layer

Event and notification layer

- Model events and Livewire actions trigger notifications
- Queue + mail deliver operational updates
- Database notifications power the bell/history surfaces
- This event layer keeps admins, SBO owners, and users synchronized

Architecturally, the app is held together by route-based entry points, shared status/state checks, and a notification system that carries changes across the roles.

Sources: docs/platform-architecture.md, routes/web.php, composer.json, package.json

Architecture distilled

This is the short version of the platform structure shown in the report.

- What it is: a connected business registry platform for discovery, SBO onboarding, procurement, support, and administration.
- How it is connected: one Laravel 12 application orchestrates routes, workflow state, data, files, and notifications.
- How roles work: Public, SBO manager, and Admin each enter through different routes but converge on the same platform core.
- How modules connect: payment state unlocks document compliance, then verification unlocks the wider SBO workspace.
- What the stack is: Laravel 12, Livewire, Volt, Blade, Tailwind CSS 4, DaisyUI, Vite, Reverb, Spatie Media Library, and queued notifications.
- Why the visuals matter: the clearest way to understand this codebase is as a set of connected journeys and module boundaries, not as isolated screens.